

一、 评估的核心维度

微调后的模型评估通常需要从以下四个维度展开：

1.特定任务能力 (Task-Specific Performance)

评估模型在微调目标领域的表现（如：医疗问答、代码生成、公文写作、信息抽取）。这是微调最直接的目的。

2.通用能力保持 (General Capabilities / 灾难性遗忘检测)

检查模型在微调后，是否遗忘了基座模型原有的常识推理、语言理解、逻辑推理等通用能力。

3.指令遵循与格式控制 (Instruction Following)

评估模型是否能严格遵守复杂的 Prompt 指令（如：字数限制、输出特定 JSON 格式、包含特定关键词）。

4.安全性与对齐 (Safety & Alignment)

评估模型是否会产生幻觉（Hallucination）、毒性输出、偏见，以及是否具备适当的拒答能力（防止过度拒答 Over-refusal）。

二、 评估方法

目前主流的评估方法分为三大类，通常建议组合使用：

1. 传统自动评估 (Automatic Metrics)

适用于有标准答案的客观任务。

文本匹配/生成：BLEU, ROUGE, METEOR（常用于翻译、摘要）。

分类/选择题：Accuracy（准确率）, F1-Score, Perplexity (PPL)。

精确匹配：Exact Match (EM)（常用于抽取式问答）。

代码生成：Pass@k（通过执行生成的代码来验证正确性，如

HumanEval) 。

2. 基于大模型的评估 (LLM-as-a-Judge)

目前最主流的开放式生成任务评估方法。使用能力更强的模型（如 GPT-4o, Claude 3.5）作为裁判，对微调模型的输出进行打分。

单点打分 (Pointwise): 直接给生成结果打 1-10 分。

成对比较 (Pairwise): 给出基座模型和微调模型的两个回答，让裁判选出更好的一个（计算胜率 Win Rate）。

优点：成本低、速度快、与人类偏好一致性较高。

缺点：存在“长度偏见”（偏好长回答）、“位置偏见”和“自我偏好”。

3. 人工评估 (Human Evaluation)

评估的“黄金标准”，但成本高昂。

方法：A/B 测试、李克特量表 (Likert Scale) 打分。

适用场景：最终版本上线前的验收、高度主观的任务（如创意写作、心理咨询）、以及用于校准 LLM-as-a-Judge 的准确性。

三、常用评估基准 (Benchmarks)

选择合适的 Benchmark 是评估的基础。以下是业界常用的基准库：

1. 通用能力与知识基准

英文：MMLU（多学科知识）、BBH（复杂推理）、HellaSwag（常识推理）。

中文：C-Eval、CMMLU、SuperCLUE、FlagEval。

2. 推理与代码基准

数学：GSM8K（小学应用题）、MATH（竞赛级数学）。

代码：HumanEval、MBPP、LiveCodeBench（防止数据污染的最新代码库）。

3. 指令遵循与对话基准

MT-Bench：评估多轮对话能力（通常用 LLM-as-a-Judge）。

AlpacaEval 2.0：评估单轮指令遵循能力（引入了长度去偏机制）。

IFEval：评估模型对严格格式指令（如“分三段”、“以某词开头”）的遵循能力。

4. 垂直领域基准

医疗：MedQA, CMB (中文医疗), PubMedQA。

法律：LawBench, JEC-QA。

金融：FinQA, FLUE。

四、 评估流程与最佳实践

1. 严防数据污染 (Data Contamination)

这是微调评估中最容易犯的致命错误。如果测试集的数据（或其变体）不小心混入了训练集，评估结果将虚高。

对策：使用最新的 **Benchmark**（如 LiveCodeBench），或在构建内部测试集时，确保训练集和测试集在时间、来源上严格隔离，并进行 **N-gram** 查重。

2. 建立对比基线 (Baselines)

评估不能只看绝对分数，必须有对比：

基座模型 (Base Model)：证明微调带来了提升。

上一版微调模型 (Previous Checkpoint)：证明本次迭代是正向的。

SOTA 闭源模型 (如 GPT-4o): 了解当前距离行业最高水平的差距。

3. 关注“灾难性遗忘”

在评估特定任务指标提升的同时，必须跑一遍 **MMLU** 或 **C-Eval** 等通用基准。如果发现特定任务提升了 **20%**，但通用能力下降了 **10%**，这种微调通常是失败的（可以通过调整学习率、使用 **LoRA** 或混合通用数据进行 **Replay** 来缓解）。

4. 深入的错误分析 (Error Analysis)

不要只看最终的 **Accuracy** 或 **Win Rate**。必须抽样查看 **Bad Cases**:

模型是在什么类型的 **Prompt** 下失败的？

是理解错了意图，还是推理过程出错，亦或是格式没遵循？

根据 **Bad Cases** 补充训练数据，进行下一轮迭代。

五、当前面临的挑战与趋势

“刷榜”与指标通货膨胀: 许多开源模型针对特定 **Benchmark** 进行了过度拟合 (**Overfitting**)，导致榜单分数高但实际体验差。趋势：转向使用动态更新的测试集（如 **LiveBench**）和基于真实用户日志的评估。

Agent 与工具调用评估: 随着大模型向 **Agent** 演进，传统的文本评估已不够用。趋势：评估模型在复杂环境中的规划能力、**API** 调用准确率以及多步执行的成功率（如 **SWE-bench**, **AgentBench**）。

细粒度/过程评估: 对于数学和代码等推理任务，不仅评估最终答案 (**Outcome**)，还要评估推理过程 (**Process**) 的正确性（如使用 **Process Reward Models, PRM**）。

总结

微调大模型的评估是一个“构建测试集 -> 自动评估 + LLM 裁判 -> 抽样人工校验 -> 错误分析 -> 优化数据/参数”的闭环过程。建立一套自动化、防污染、多维度的评估 Pipeline，是保障大模型高质量落地的核心基础设施。